

Computer vision

Image segmentation- traditional
approaches

Vanda Benešová

Type of task



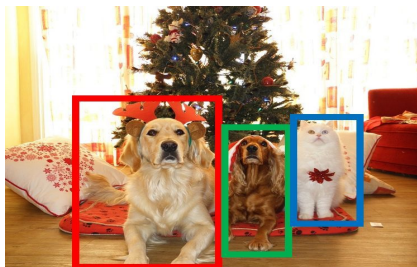
Object detection



Object segmentation



Object detection and recognition



DOG, DOG,
CAT



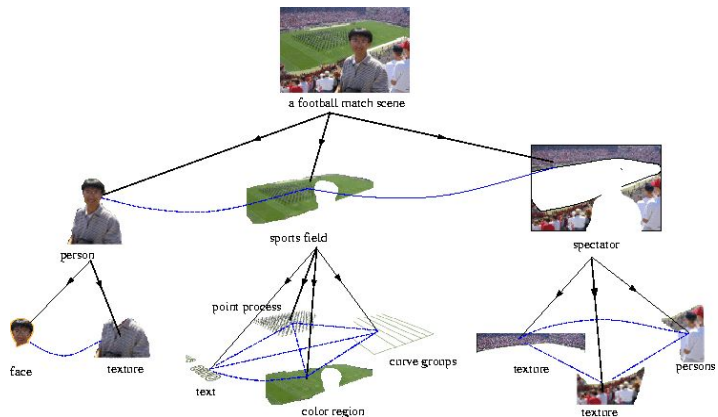
DOG, DOG,
CAT

Multiple Object

This image is [CC0 public domain](#)

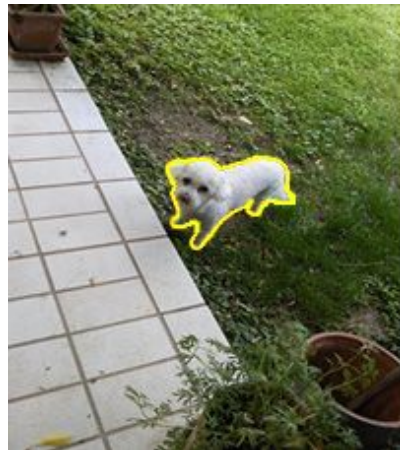
Complete segmentation/ Partial segmentation

Semantic Segmentation



Complete segmentation

Label each pixel in the image with a class label



Partial segmentation

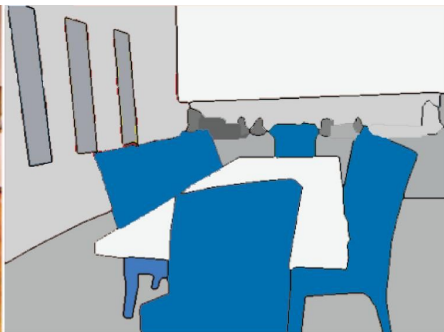
Semantic segmentation

Semantic labeling / pixel classification

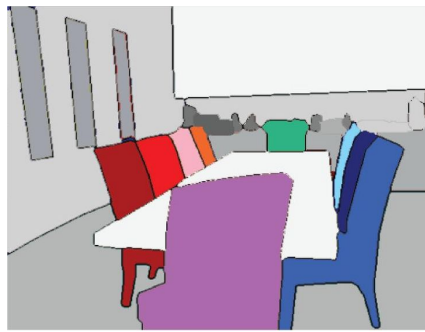
Example: chairs



Input Image



Semantic Segmentation



Semantic Instance
Segmentation

Segmentation methods

- Region based methods
 - Thresholding
 - Growing regions
 - Clustering (unsupervised classification)
 - Split/Merge
 - Morphological methods – watershed
- Edge-based methods:
 - Snake Model
 - Graph-cut
- Knowledge-based
- Template matching
- Statistical methods
- Artificial intelligence -> Deep Neural Networks

Segmentation is a complex task

Different tasks → specific methods

Why it is so difficult ?

Important factor – quality of the image

- Complex shapes of the objects
- Overlapping objects
- Shadows... and more

Pre-processing ?

Post-processing?

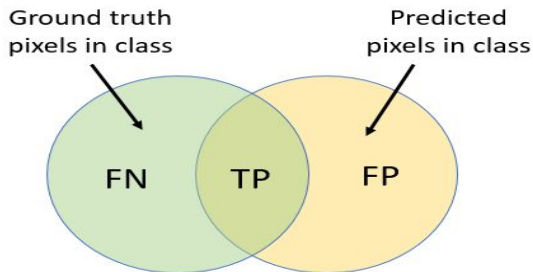
Additional knowledge? -> Whenever additional knowledge is available for segmentation - **it should be used!**

Evaluation metrics for segmentation task

True/False Positives/Negatives

Confusion matrix:

		Predicted Class	
		P	N
Actual Class	P	True Positives (TP)	False Negatives (FN)
	N	False Positives (FP)	True Negatives (TN)



Overlap based metrics

measure of the similarity between two sets (partitions, segments):

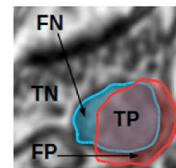
S_p - predicted points and
 S_l - labeled points

The Dice coefficient (DICE): $2 \frac{|S_p \cap S_l|}{|S_p| + |S_l|} = \frac{2TP}{2TP + FP + FN}$

The Jaccard index (J, JAC)

is defined as the size of the intersection divided by the size of the union of the sample sets

$$JAC = \frac{|S_p \cap S_l|}{|S_p \cup S_l|} = \frac{TP}{TP + FP + FN}$$



Jacc = $\frac{\text{Intersection}}{\text{Union}}$

Thresholding

Thresholding -> for simple segmentation tasks

Contrasted objects on a uniform background

simple assembly tasks:

- blood cells segmentation
- printed characters segmentation... etc.

Global thresholding

In global thresholding, the threshold value is held constant throughout the whole image

Global thresholding is of the form:

$$g(x,y) = \begin{cases} 0 & f(x,y) < \text{Threshold} \\ 1 & f(x,y) \geq \text{Threshold} \end{cases}$$

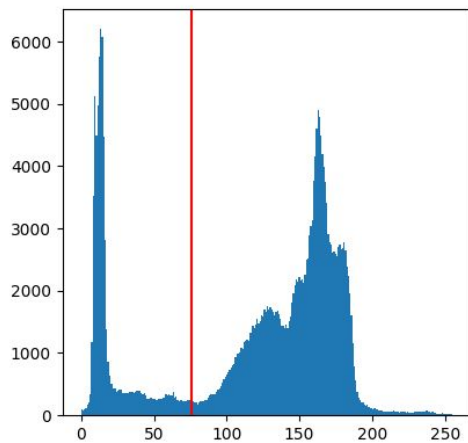
Output of thresholding -> binary mask

Thresholding

Threshold value?

Bimodal histogram

Original



Thresholded



Threshold value derived from bimodal histogram

Otsu's method

Otsu's thresholding chooses the threshold to **minimize the intraclass variance and maximize the interclass variance** between the segmented foreground (object) and background.

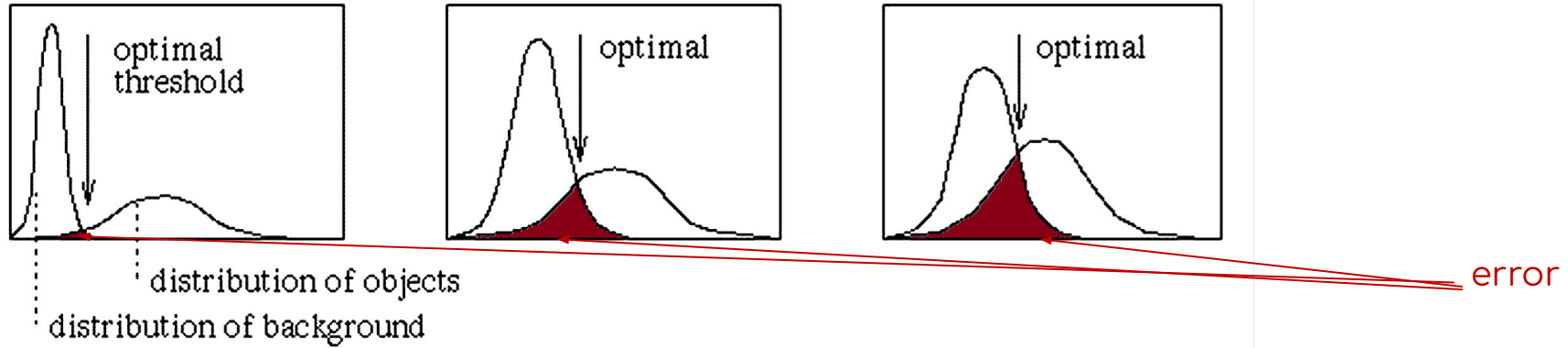
Algorithm calculation:

- All possible thresholds (0-255) are evaluated.
- The threshold that **minimize the intraclass variance and maximize the interclass variance** criterion is chosen as the optimal threshold.

Otsu'su thresholding - examples



Optimal threshold



Probability distributions of background and object
→ optimal threshold results in minimum error segmentation

(Discrete probability distribution is represented by histogram)

Gaussian Mixture Models (GMM)

Mixture of Gaussians (MOG)

probability **model** estimation $\rightarrow$ we can suppose to have mixture of **Gaussian distribution**

$$f(x) = \sum_{i=1}^k p_i N(x|\mu_i, \sigma_i^2)$$

Where k is the number of regions and $p_i > 0$ are weights such that

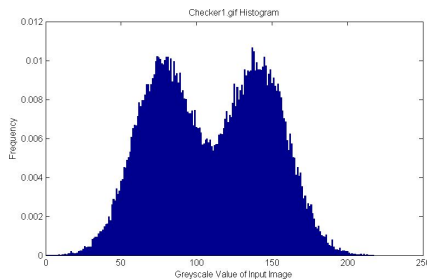
$$\sum_{i=1}^k p_i = 1$$

$$N(\mu_i, \sigma_i^2) = \frac{1}{\sigma_i \sqrt{2\pi}} \exp \frac{-(x - \mu_i)^2}{2\sigma_i^2}$$

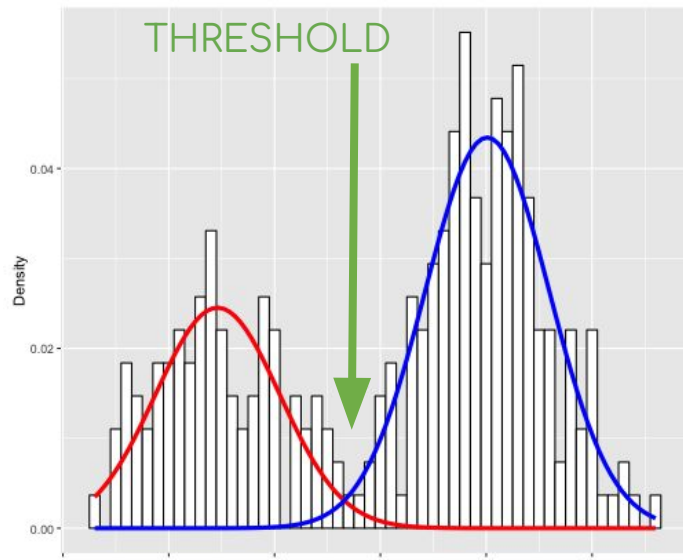
Where μ_i, σ_i are mean resp. standard deviation of class i .

Examples : Gaussian Mixture Models (GMM) Mixture of Gaussians (MOG)

1. Calculate histogram
2. Choose the number of Gaussian components (k)
3. Modeling the histogram with a Mixture of Gaussians:
Fitting the model - typically done using Expectation-Maximization (EM)



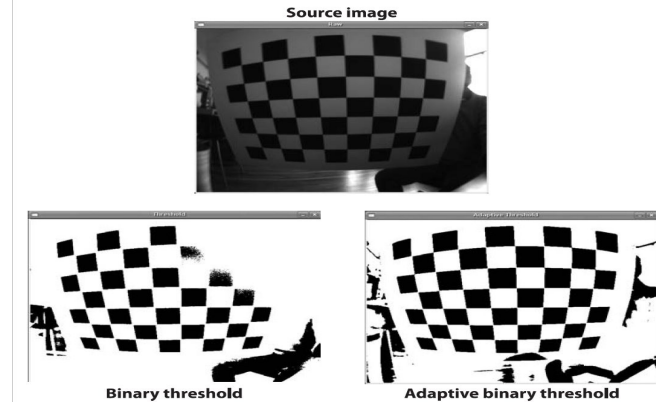
$k=2$



Global, Local Threshold, Multimodal thresholding

Global / local / adaptive thresholding

- **Global thresholding** -> the threshold value is held constant throughout the image
- **Local thresholding / Adaptive thresholding** - technique in which the threshold value is variable.
-> partially eliminate changes in illumination



ρ -tile “Percentile” thresholding

p-tile “Percentile” thresholding

Using prior knowledge

This method requires knowledge about the area or size of the objects which should be segmented.

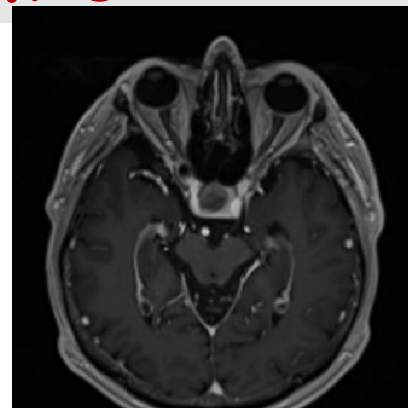
P-tile means “Percentile”, and the threshold is chosen to be the intensity value where the cumulative sum of pixel intensities is closest to the percentile.

Example : a printed text sheet if we know that characters of the text cover approx. $1/p$ of the sheet area.

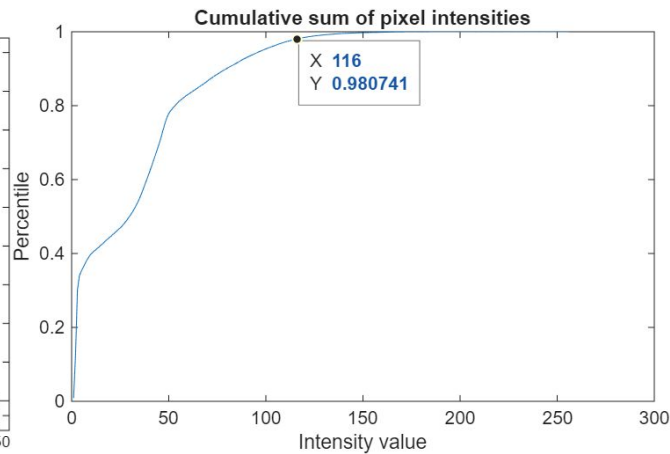
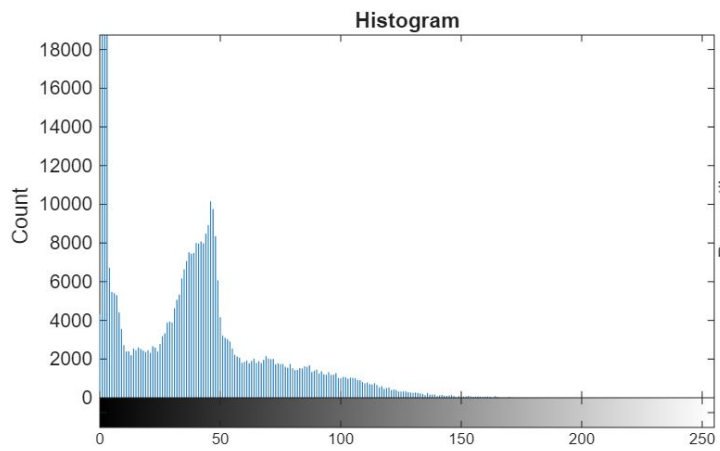
p-tile thresholding : example using percentile 0.98

Threshold value: 116

Input image:



Threshold = 116



Segmentation based on edge detection

Segmentation based on edge detection

Segmentation based on edge detection:

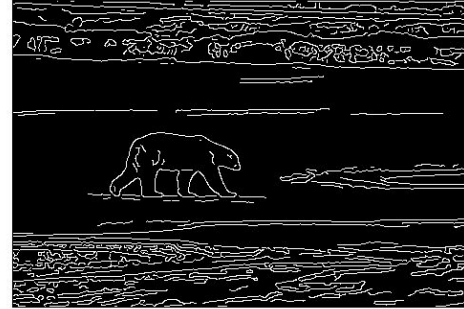
Determination of object boundaries from a edge image (Sobel, Laplace, Canny detector...)

Problems:

- Edges are overlapped, not continuous...
- Non relevant edges have to be avoid (included in Canny detector)
- Pre-processing and post-processing could be necessary...

Examples - Results of Canny edge detection

Canny algorithm: Alternatively, non-maximal suppression and hysteresis thresholding can be used as was introduced in the Canny edge detector.



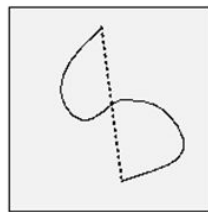
Active contour model- snake

Active contour model- snake

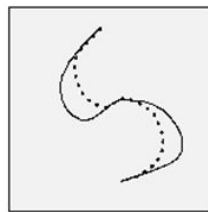
The active contour model- snake, is defined as an **energy minimizing spline** - the snake's energy depends on its shape and location within the image.

We need two images on the input:

- a) input grayscale image
- b) initial snake position



(a)



(b)



(c)

The goal of the iterative process is to minimize the snake energy

Parametric Snake Model

- The total energy of the snake is defined as

$$E_{\text{total}} = E_{\text{internal}} + E_{\text{external}}$$

Encourages smoothness or
a particular shape



Encourages curve onto
image structures (eg edges)



Snake Model - Internal Energy

For a curve represented as a set of points.

- The elastic (length) energy can be expressed as

$$E_{in_elasticity} = \alpha \sum_{i=0}^{n-1} L_i^2$$
$$= \alpha \sum_{i=0}^{n-1} (x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2$$

*where X_i and Y_i are snake points coordinates,
 α is multiplicative constant*

Snake Model - External Energy

The external energy describes how well the curve matches the image data locally.

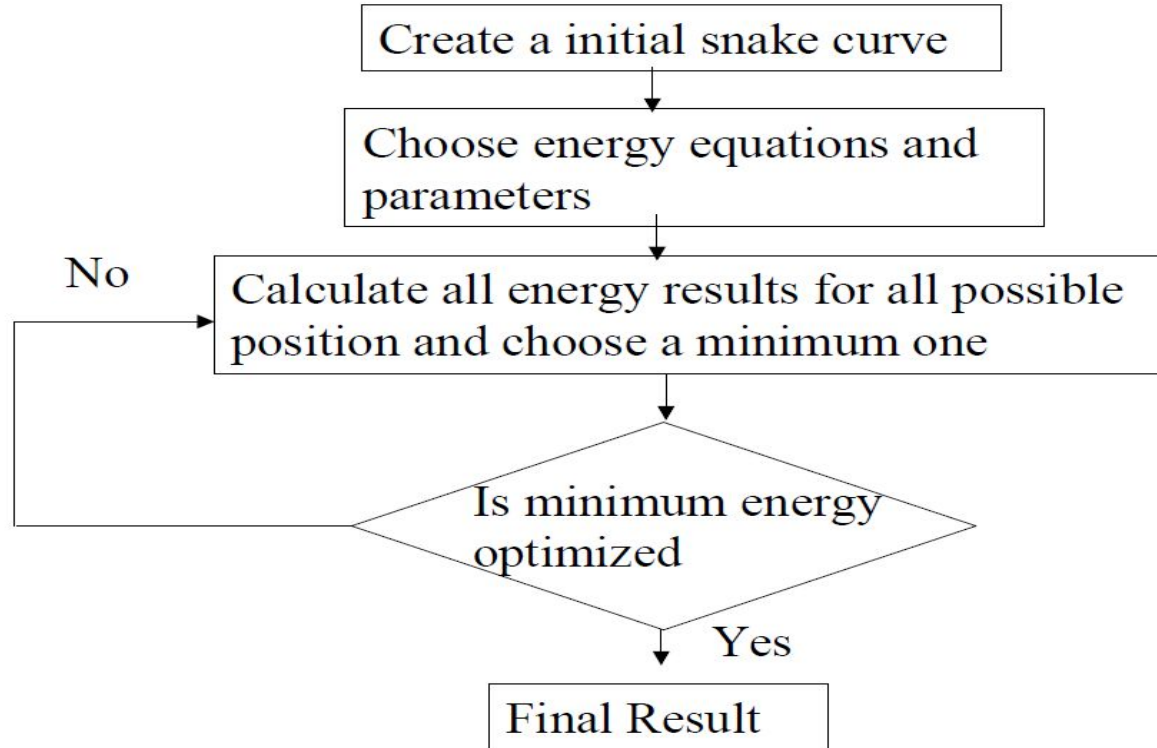
Numerous forms can be used, attracting the curve toward different image features.

Gradient energy form:

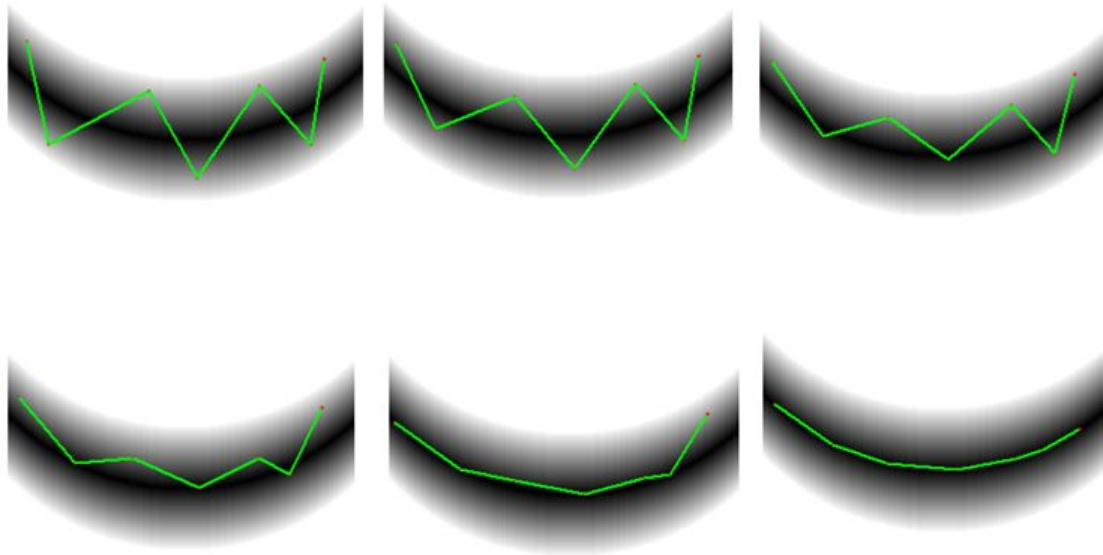
$$E_{ext_grad} = -\lambda_1 \sum_{i=0}^{n-1} (|G_x(x_i, y_i)|^2 + |G_y(x_i, y_i)|^2)$$

*where G_x and G_y are gradients at the snake X_i and Y_i positions
 λ_1 is multiplicative constant*

Snake Model - Flow Chart



Snake Model - Demo Pictures

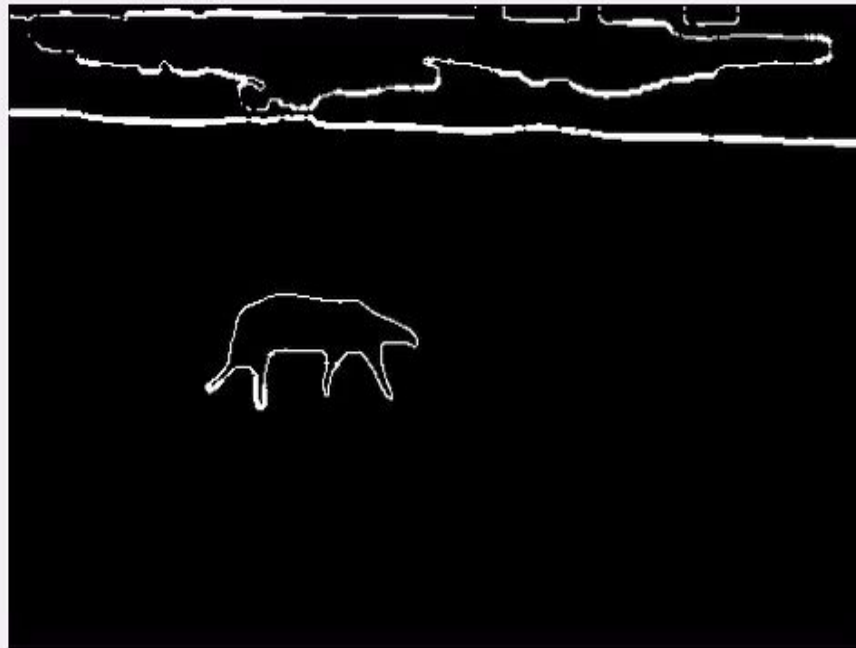
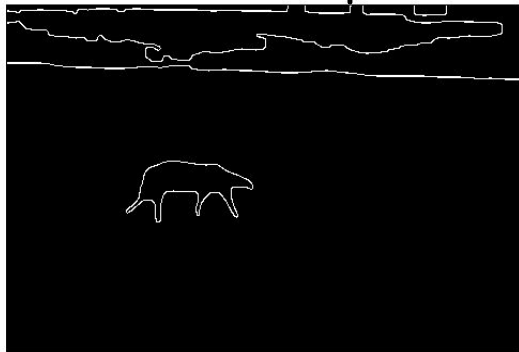


Snake example

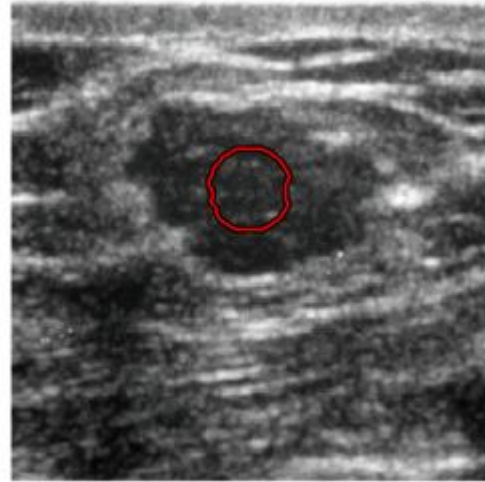
Input grayscale image



initial snake position



Snake Model - Demo



Hough Transforms

Hough Transforms

The Hough transform is a method for finding lines, circles, or other simple forms in an image.

The purpose of the technique is to find instances of objects within a certain class of shapes by a **voting procedure**.

This voting procedure is carried out in a parameter space – accumulator.

Hough Line Transform

The underlying principle of the Hough transform is that there are an infinite number of potential lines (*přímek*) that pass through any point, each at a different orientation.

The purpose of the transform is to determine which of these theoretical lines pass through most *points* in an image - that is, which lines fit most closely to the data in the image

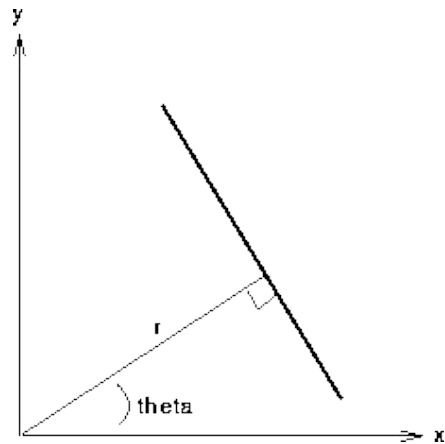
Hough Transform

In the standard Hough transform, each line is represented by two parameters, commonly called r and θ (*theta*), which represent the length and angle from the origin of a normal to the line.

By transforming all the possible lines through a point into this coordinate system - i.e. calculating the value of r for every possible value of θ

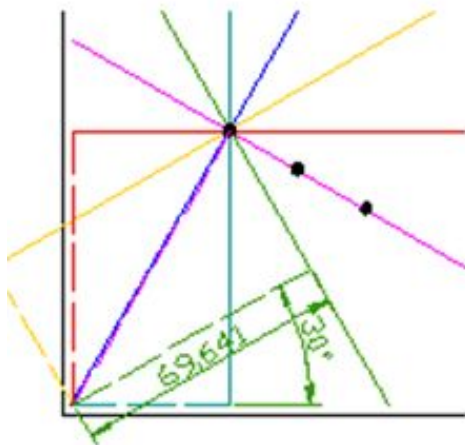
This representation of the two parameters is sometimes referred to as *Hough space*.

$$r = x \cdot \cos(\theta) + y \cdot \sin(\theta)$$

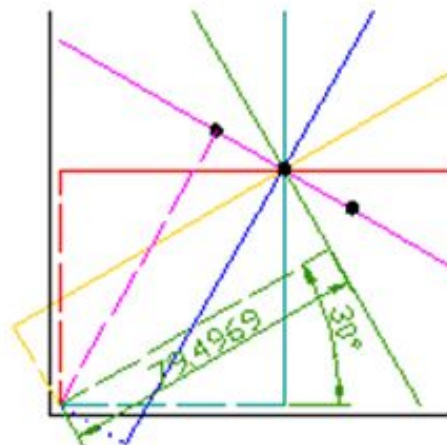


Hough Line Transform

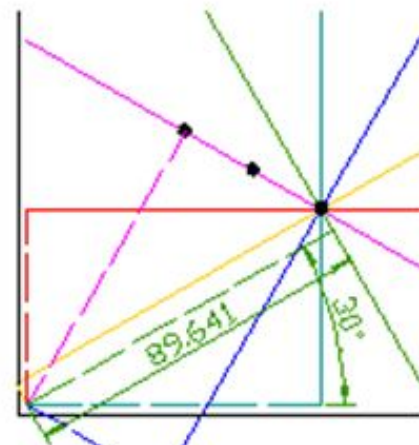
Consider three data points, shown here as black dots.



Angle	Dist.
0	40
30	69.6
60	80.5
90	70
120	40.6
150	0.4



Angle	Dist.
0	57.1
30	79.5
60	80.5
90	60
120	23.4
150	-19.5



Angle	Dist.
0	74.6
30	89.6
60	80.5
90	50
120	6.0
150	-39.6

Hough Line Transform

For each data point, a number of lines are plotted going through it, all at different angles. These are shown here as solid lines.

For each solid line a line is plotted which is perpendicular to it and which intersects the origin. These are shown as dashed lines.

The length and angle of each dashed line is measured. In the diagram above, the results are shown in tables.

This is repeated for each data point.

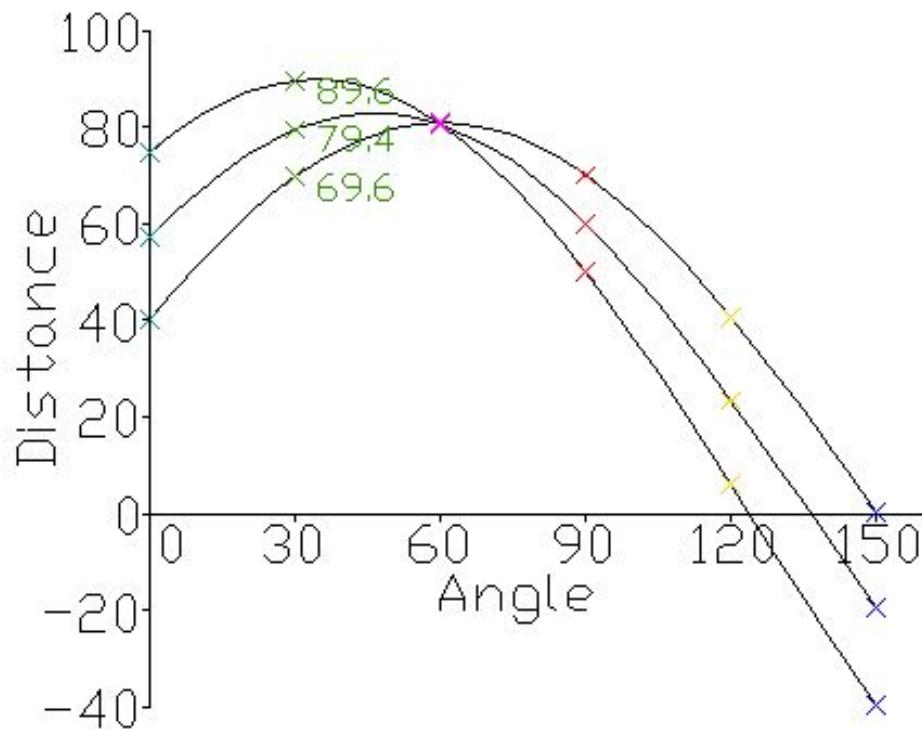
A graph of length against angle, known as a Hough space graph, is then created.

Hough Line Transform

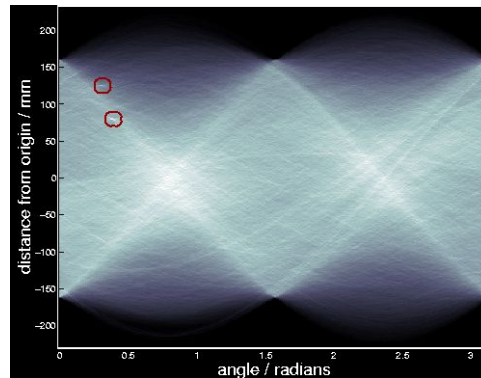
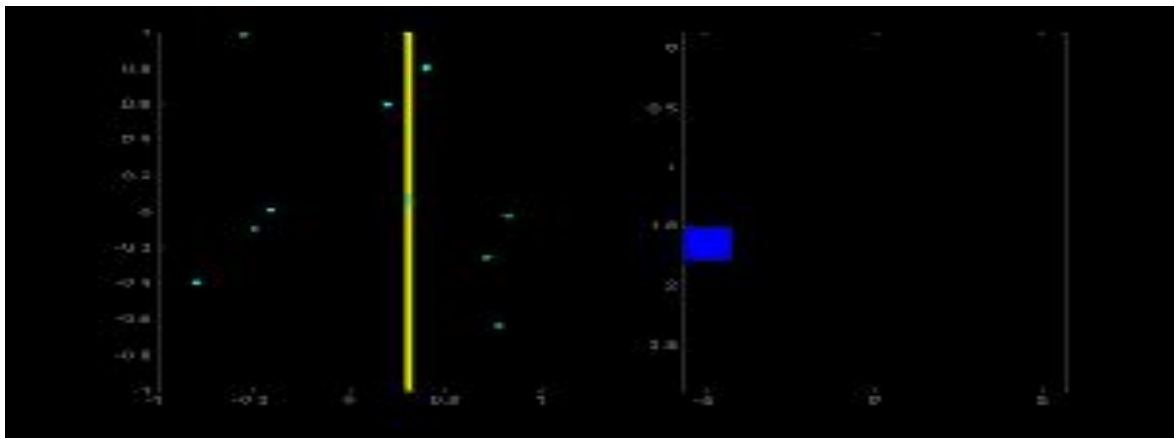
The point where the lines intersect gives a distance and angle. This distance and angle indicate the line which bisects the points being tested.

In the graph shown the lines intersect at the purple point; this corresponds to the solid purple line in the diagrams above, which passes through all three points.

Hough space plot example:

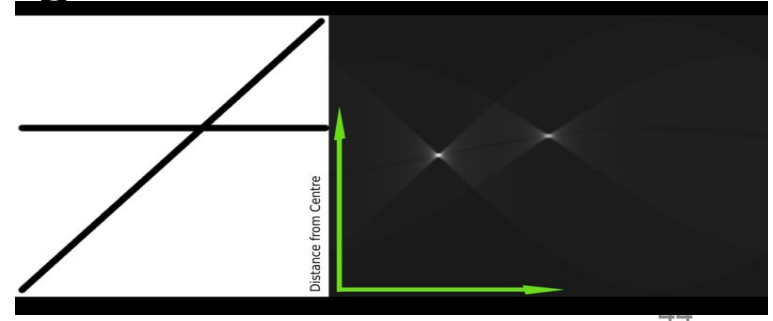


Hough Line Transform - demo



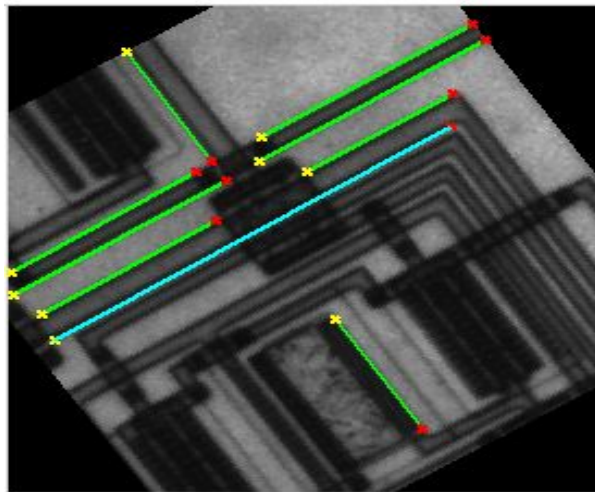
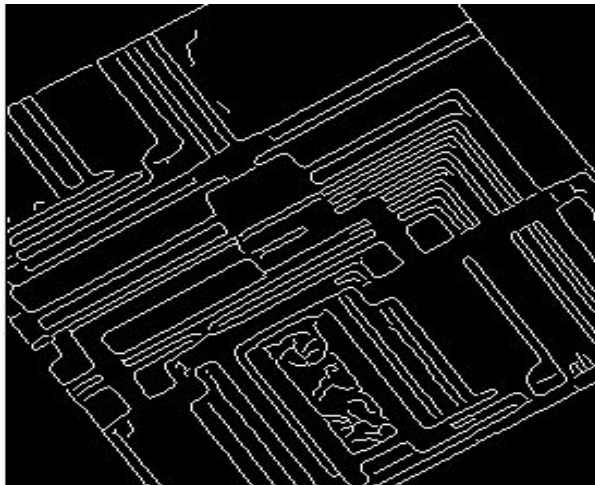
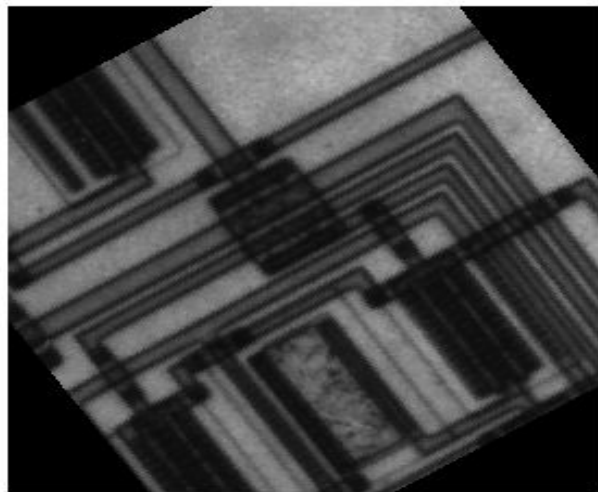
Example : raster image containing two thick lines

- The results of this transform were stored in a matrix.
- Cell value represents the number of curves through any point. Higher cell values are rendered brighter.
- The two distinctly bright spots are the Hough parameters of the two lines. From these spots' positions, angle and distance from image center of the two lines in the input image can be determined.



Hough Transform - example

Highlight the longest line segments:



Hough transform of curves

A similar transform can be used for finding **any shape which can be represented by a set of parameters.**

A circle, for instance, can be transformed into a set of three parameters, representing its centre and radius, so that the Hough space becomes three dimensional.

Detection of 3D objects (Planes and cylinders)

Hough transform can also be used for the detection of 3D objects in range data or 3D point clouds.

The extension of classical Hough transform for plane detection is quite straightforward. A plane is represented by its explicit equation

$$z = a_x * x + a_y * y + d$$

for which we can use a 3D Hough space corresponding to a_x , a_y and d .

This formulation of the plane has been used for the detection of planes in the point clouds acquired from airborne laser scanning and works very well because in that domain all planes are nearly horizontal.

Hough Transform

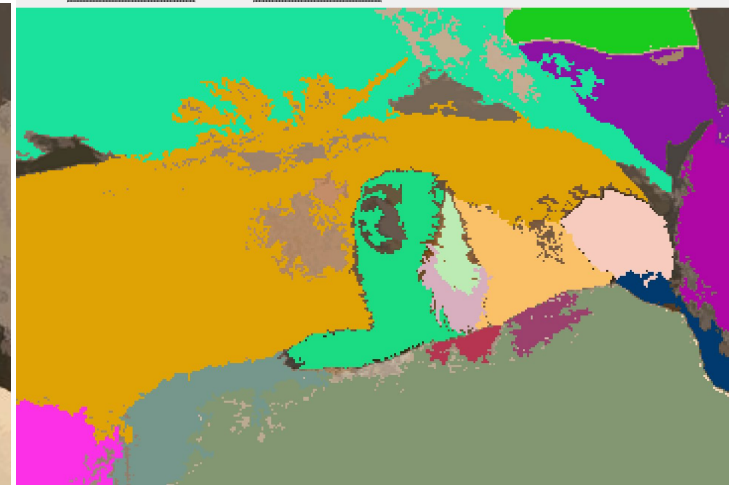
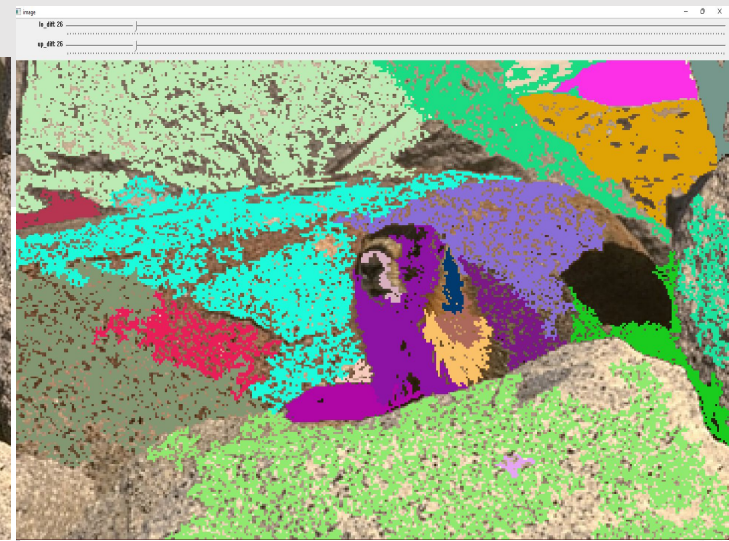
A *classical* Hough transform is most commonly used for the detection of regular curves such as lines, circles, ellipses, *etc.*

A *generalized* Hough transform can be employed in applications where a simple analytic description of a feature(s) is not possible

https://www.youtube.com/watch?v=_mGxmZWs9Zw

Region based methods

An example of growing regions



Demo OpenCv

Growing regions

Preprocessing could be beneficial!



Growing regions

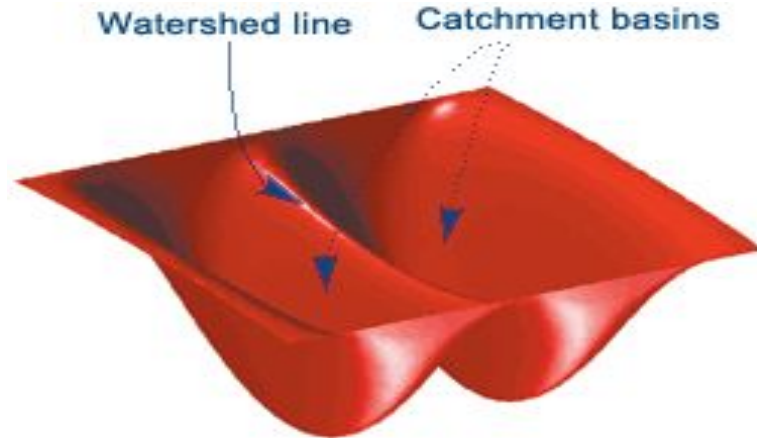
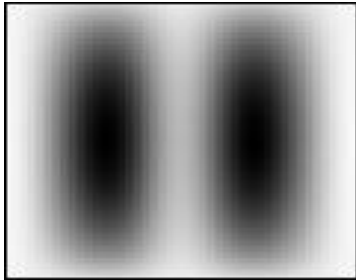
- Select a start (seed) point
- Grow the point based on a certain property of homogeneity (Connectivity should be considered)
- Seed point selection:
 - for example - highlighted point (Due to specific property)

(openCv: FloodFill)

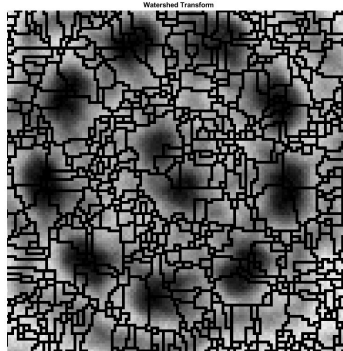
Watershed segmentation

Region based method- Watershed Segmentation

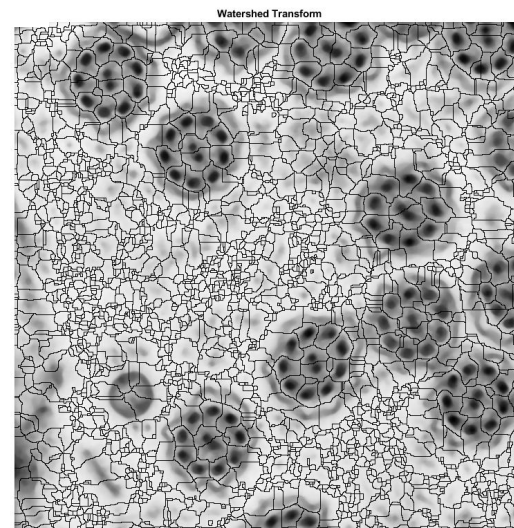
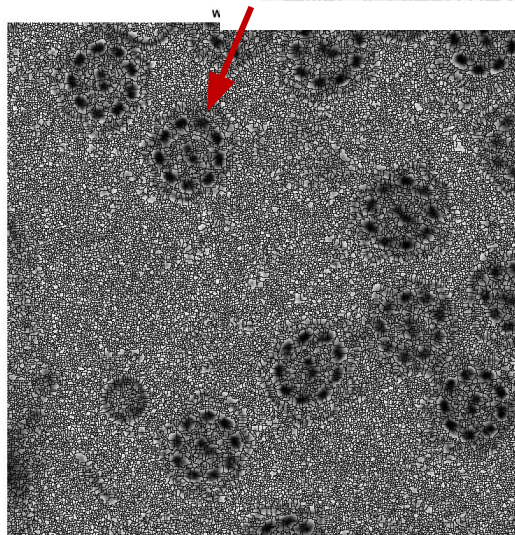
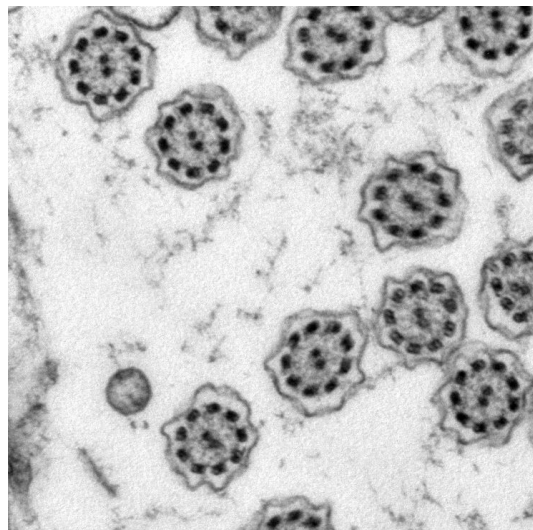
The watershed transform treats the intensity as a function defining 'hills' and 'valleys' and attempts to detect the valleys.



Watershed example



WS after median 9x9
filtration



Segmentation by clustering

Region based methods using clustering

Segmentation may be regarded as spatial clustering:

- clustering of selected features (color..)

and

- spatial connected components

Clustering - unsupervised classification

Consider segmentation as a classification problem

An image consists of K regions. Each pixel have to be classified into the one of this regions.

- K-Means algorithms

- Mean Shift algorithms

- Fuzzy C-Means

- Gaussian Mixture Models

- and much more...

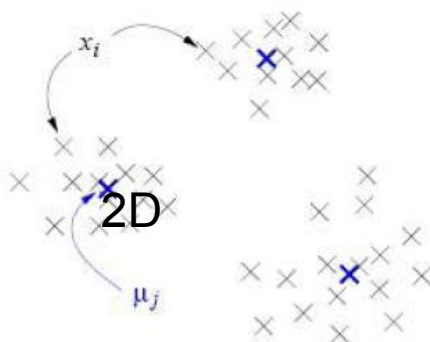
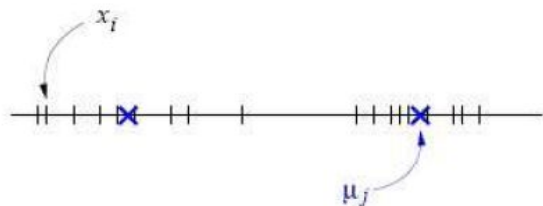
K-Means Clustering

K-Means Algorithm

Each region is represented by a mean value – μ_j .

Minimum distance classification

$$O = \sum_{k=1}^K \sum_{\mathbf{x} \in C_k} \|\mathbf{x} - \boldsymbol{\mu}_k\|.$$



K-Means Algorithm

1. Randomly initialize centres of regions μ_i .
2. Classify each vector x_i to the nearest cluster:

$$y_i = \arg \min_{k \in \{1, \dots, K\}} \|x_i - \mu_k\|^2$$

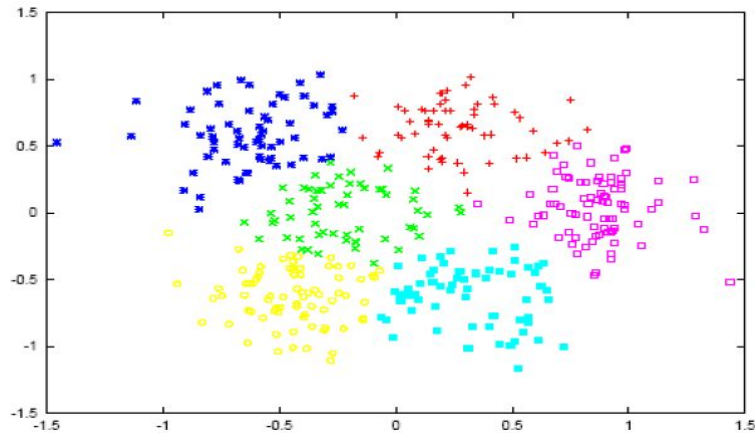
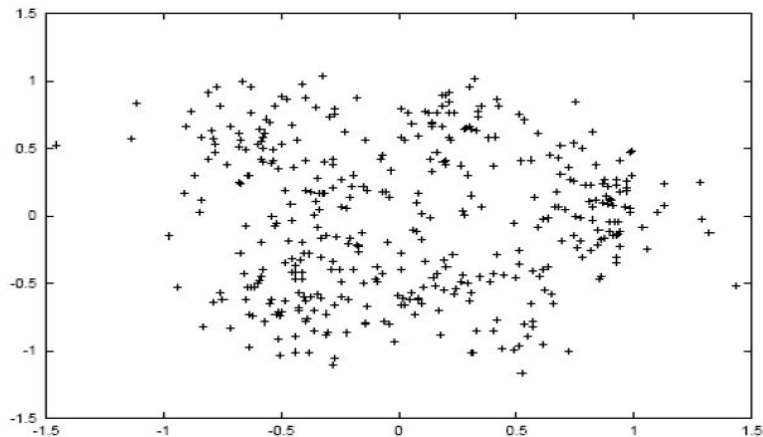
$$\mu'_k = \frac{1}{N_{y_i=k}} \sum_{i: y_i=k} x_i$$

3. Calculate a new centres of regions

Repeat steps (2) and (3) until the center of the cluster changes.

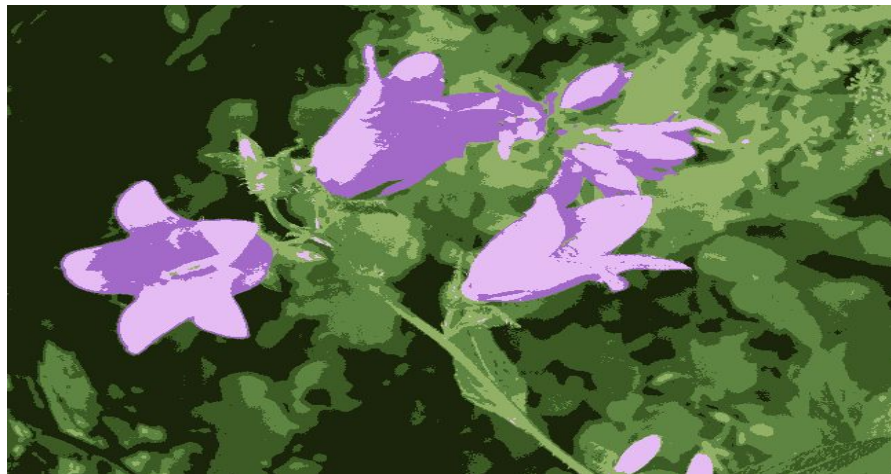
K-means Algorithm

Dividing of the data into compact clusters.



K-Means Clustering

K=6 in 3-Dim feature space: RGB color space:



K-Means

K=5 in 3-Dim feature space: RGB color space:



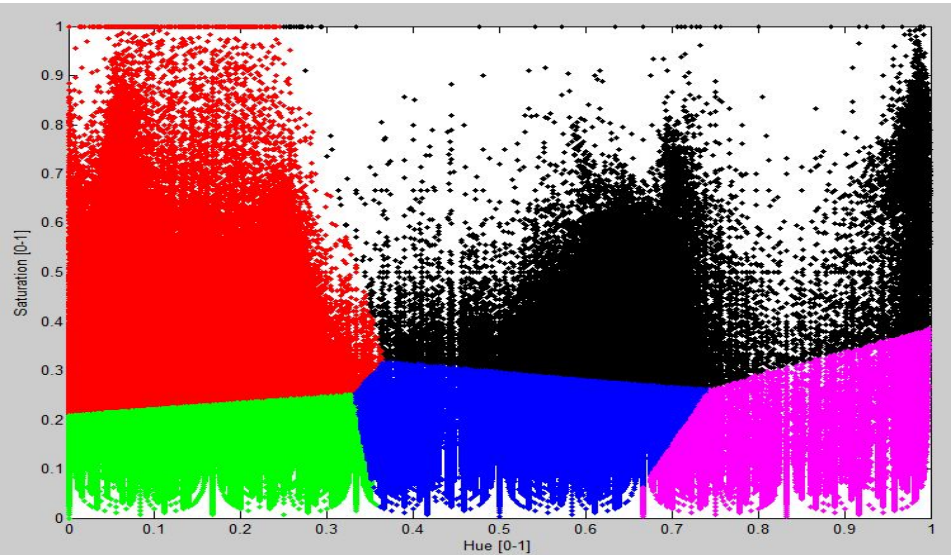
K-Means

K=5 in 2-Dim feature space: Hue, Saturation



K-Means - Visualisation of the 2Dim feature space

K=5 in 2-Dim feature space: Hue, Saturation



Split & Merge Algorithm

Split & Merge Algorithm

- Recursively splits an image region (which starts out as the entire image) by dividing it into quarters.
- Feature vectors are computed for each block.
- When we are finished, there will be many different sized blocks, each of which has a homogeneous texture according to our model.

However, the arbitrary division of the image into rectangles (called a „quad-tree" decomposition) might have accidentally split up regions of homogeneous texture.

- The merge step, then, tries to fix this by looking at adjacent regions that were not compared to each other during the split phase, and merging recursively splits an image region (which starts out as the entire image) by dividing it into quarters.

Split & Merge Algorithm

Step 1 – SPLIT

SPLIT:

Split the region in 4 blocks

- If the four blocks are judged to be similar, they are grouped back together and the process ends.
- If not, each of the blocks are recursively splitted and analyzed using the same procedure.

Split & Merge Algorithm

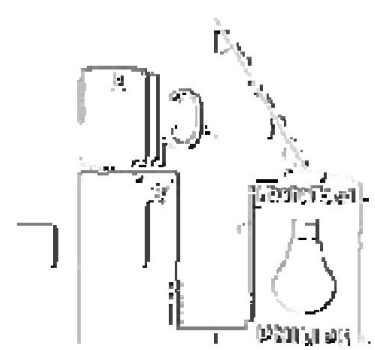
Step 2 - MERGE

Merge:

Judge for merging the neighboring splitted regions

- If the merged blocks are judged to be similar, they are grouped back together.

Split & Merge -Example



R_1	R_2	
R_3	R_{41}	R_{42}
	R_{43}	R_{44}

Split & Merge -Examples



Min size 10x10 pixelov



Supervoxel segmentation

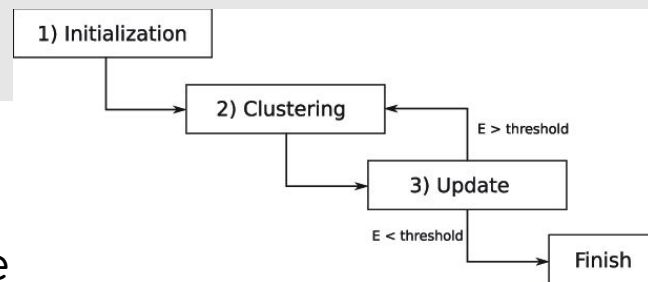
Supapixel segmentation

- results of an image over-segmentation
- regions in an image which can be used as basic units (primitives) in the next image processing



- distributed regularly with respect to the nature of the input image,
- the desirable variation of the size of superpixels is preferably small
- the boundary of superpixels has to be corresponding with the natural boundary of objects presented on the image

Simple Linear Iterative Clustering SLIC - algorithm



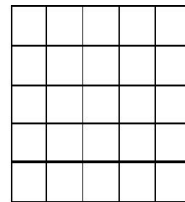
1. Initialization

Convert color images in the CIE $L^*a^*b^*$ color space

initialization step where k initial cluster centers C_i are sampled on a **regular grid** spaced S pixels apart.

Feature vector:

$$C_i = [L^* a^* b^* x y]$$

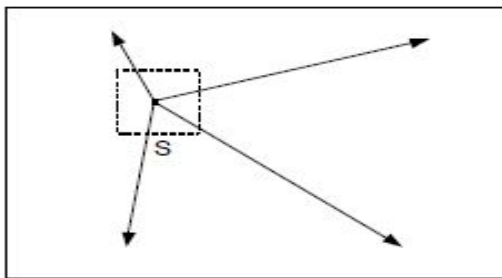


Avoid the cluster center position on a edge -> in this case the centers C_i are moved to seed locations corresponding to the lowest gradient position in a 3×3 neighborhood.

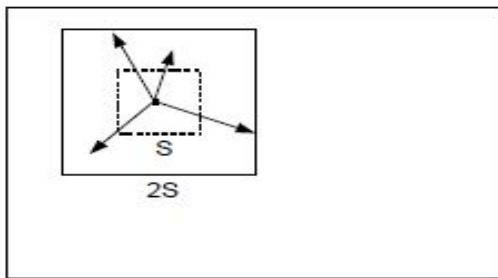
Simple Linear Iterative Clustering - SLIC - algorithm

2. Iterative clustering and update the cluster centers

each **pixel** is associated to the **nearest cluster center** (Euclidean distance of feature vector $[L * a * b * x * y]$)



(a) standard k -means searches the entire image



(b) SLIC searches a limited region

update step -> **adjusts the cluster centers** to be the mean $[L * a * b * x * y]$ vector of all the pixels belonging to the cluster using the **Euclidean distance**

Simple Linear Iterative Clustering - SLIC - algorithm

4. Finish

compute a residual error E between the new cluster center locations and previous cluster center locations.

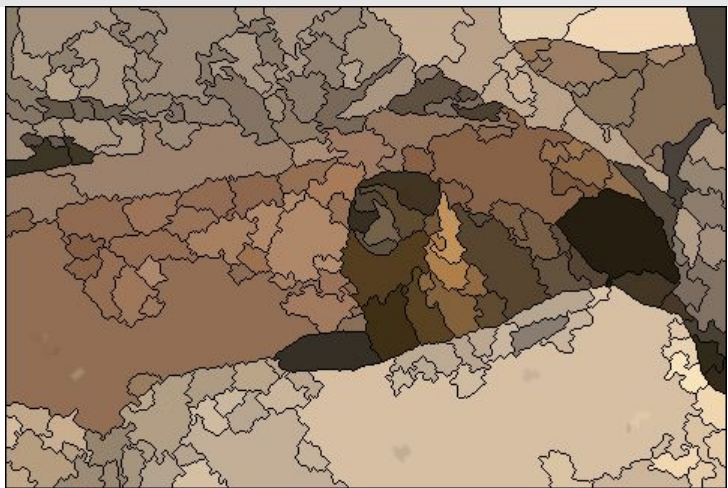
- The assignment and update steps can be repeated iteratively until the error converges, (10 iterations suffices for most images)
- Post-processing step enforces connectivity by re-assigning disjoint pixels to nearby superpixels – enforce connectivity.

Superpixels example

Demo
OpenCv



Merge superpixels

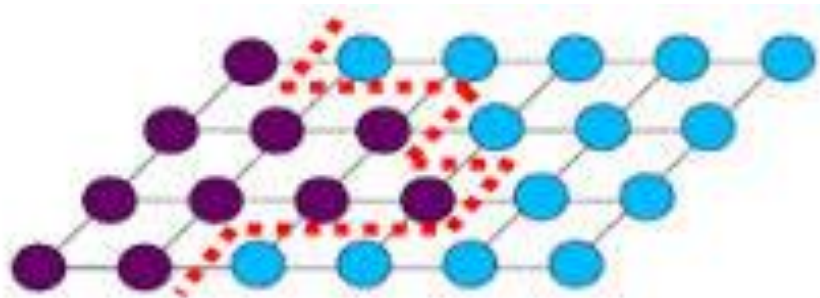


Graph-cut

Graph-cut

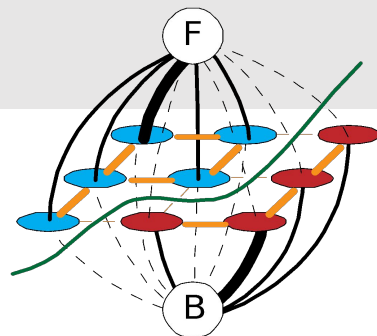
Image is modeled as a **graph** where **pixels are represented as nodes**. These nodes are connected by edges, with the weights of these **edges representing the relationships between pixels** (e.g., similarity or dissimilarity).

The goal is to find a minimum "cut" through the graph, which separates the nodes (pixels) into different segments (an object and its background) by minimizing the total (sum) weight of the edges that are cut.



Graph min cut algorithm

- Graph-cut divides the graph into F-foreground from B-background
- Minimize the sum of cut edge weight



Graph-cut is an algorithm that finds a globally optimal segmentation solution us

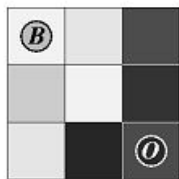
- Min-cut
- Normalized cut (Energy optimization equivalent to graph Min-cut.)

See:

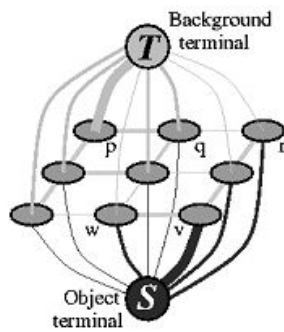
The Boykov-Kolmogorov max-flow algorithm (or often BK max-flow) in boost library

:

Seeds in the flow graph-cut



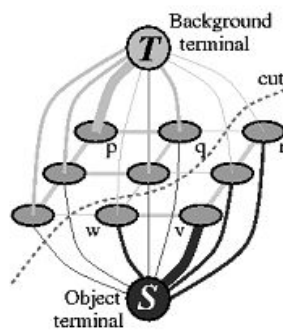
(a) Image with seeds.



(b) Graph.



(d) Segmentation results.



(c) Cut.

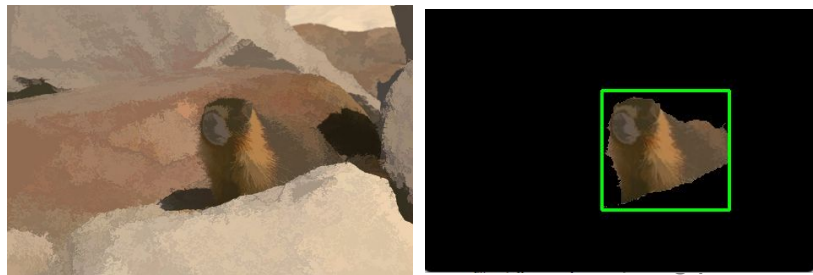
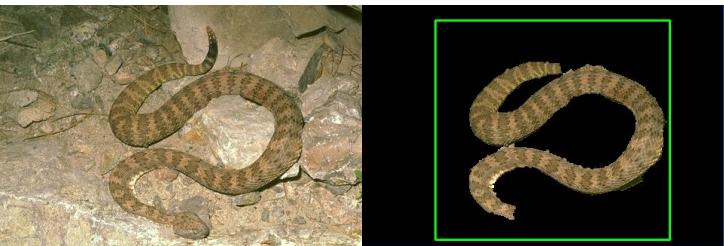
Interactive Segmentation application using Graph Cuts

User marks some pixels as seeds of “Background” and “Object”, and runs the application several times until a satisfactory result achieved.

Interactive inputs:

“seeds” pixels for “Background” and for “Object (Foreground)”

Energy function (Intensities, distances, textures, etc..)



Interactive Segmentation using Graph Cuts

Motivation: for some applications:

Fully automatic segmentation does not give sufficient results.

for example, in medical diagnostics, :

doctors' handlings a medical scan images where soft tissues, blood vessels, and bones are “blended” together , are very difficult to segment.

Example:

openCV examples

